



**Bharat Ratna Indira Gandhi College of
Engineering, Solapur**

Master of Computer Application

SUBJECT:

FULL STACK DEVELOPMENT (MCAC301)

SEMESTER-III

UNIT 6: PERFORMANCE OPTIMIZATION

A. Front-End Performance Optimization

Definition: Improving the speed and efficiency of the client-side of a web application to enhance user experience.

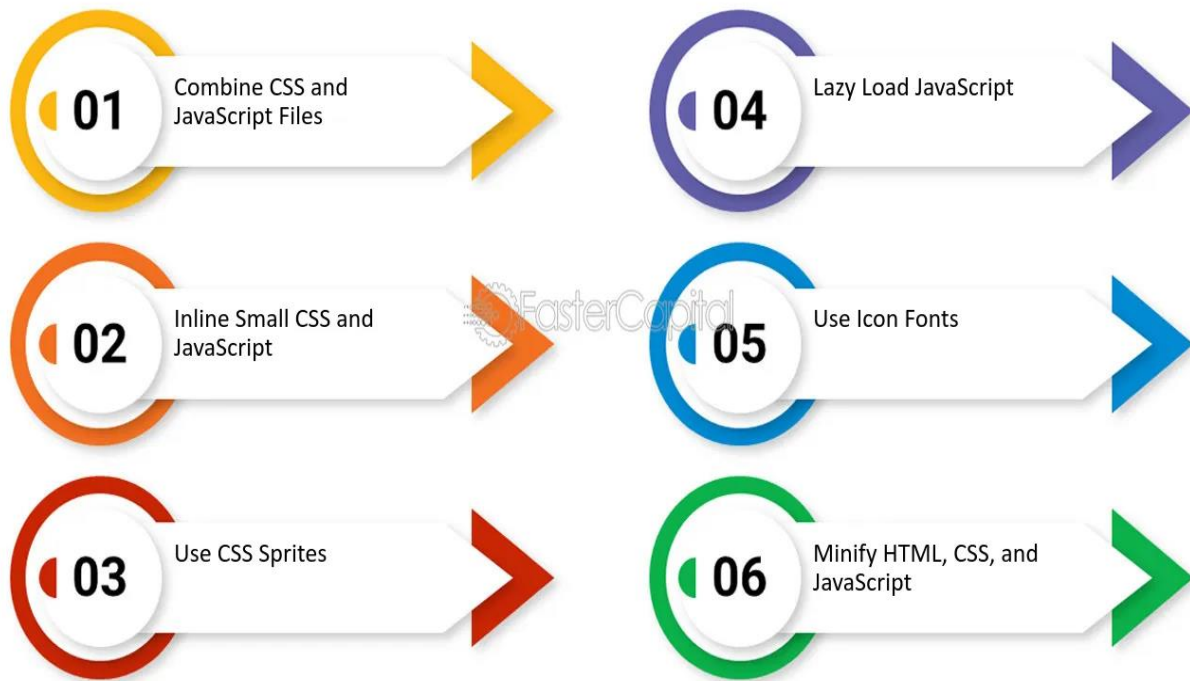


Key Areas to Focus On:

1. Minimize HTTP Requests:

Minimizing HTTP requests is essential for optimizing website performance. To achieve this, combining and minifying CSS and JavaScript files reduces the number of external requests. Image sprites and lazy loading ensure that images are efficiently loaded as users scroll down the page.

Minimizing HTTP Requests on Your Website



➤ **Objective:** Reduce the number of requests made to the server.

➤ **Strategies:**

- Combine multiple CSS files into one.
- Combine multiple JavaScript files into one.
- Use image sprites to combine multiple images into a single image file.

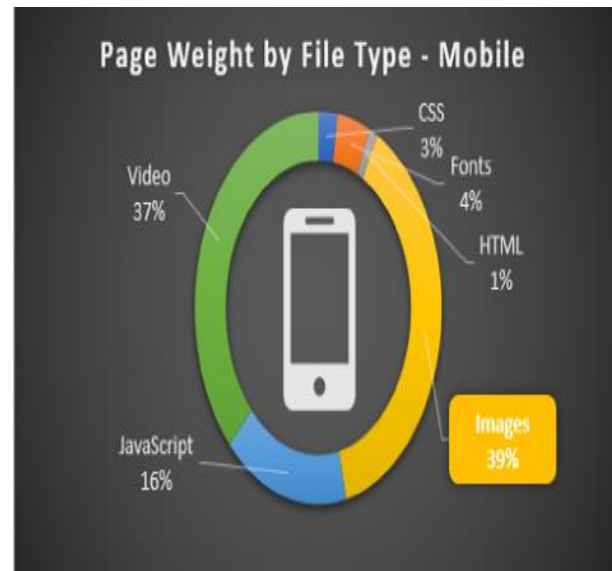
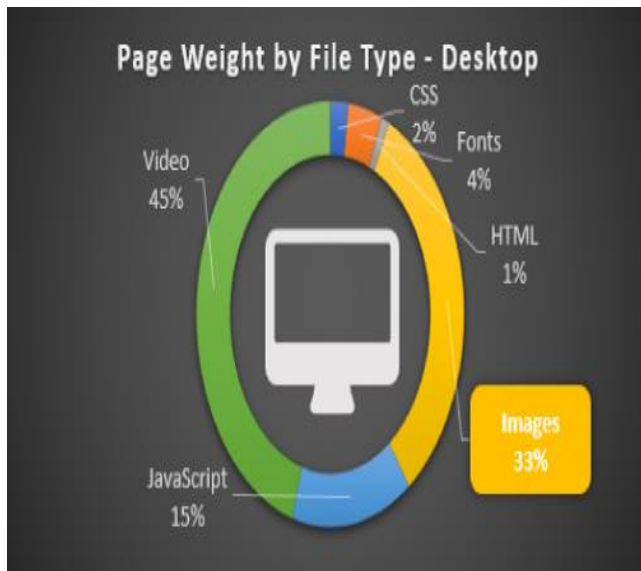
2. Optimize Images:

Optimizing an image involves delivering images in the format, dimensions, resolution, and quality that will yield the smallest possible file size while ensuring that the resulting image is appropriate for the specific content, the overall page design, the requesting device, and the expectations of your site visitors.

➤ **Objective:** Reduce the size and load time of images.

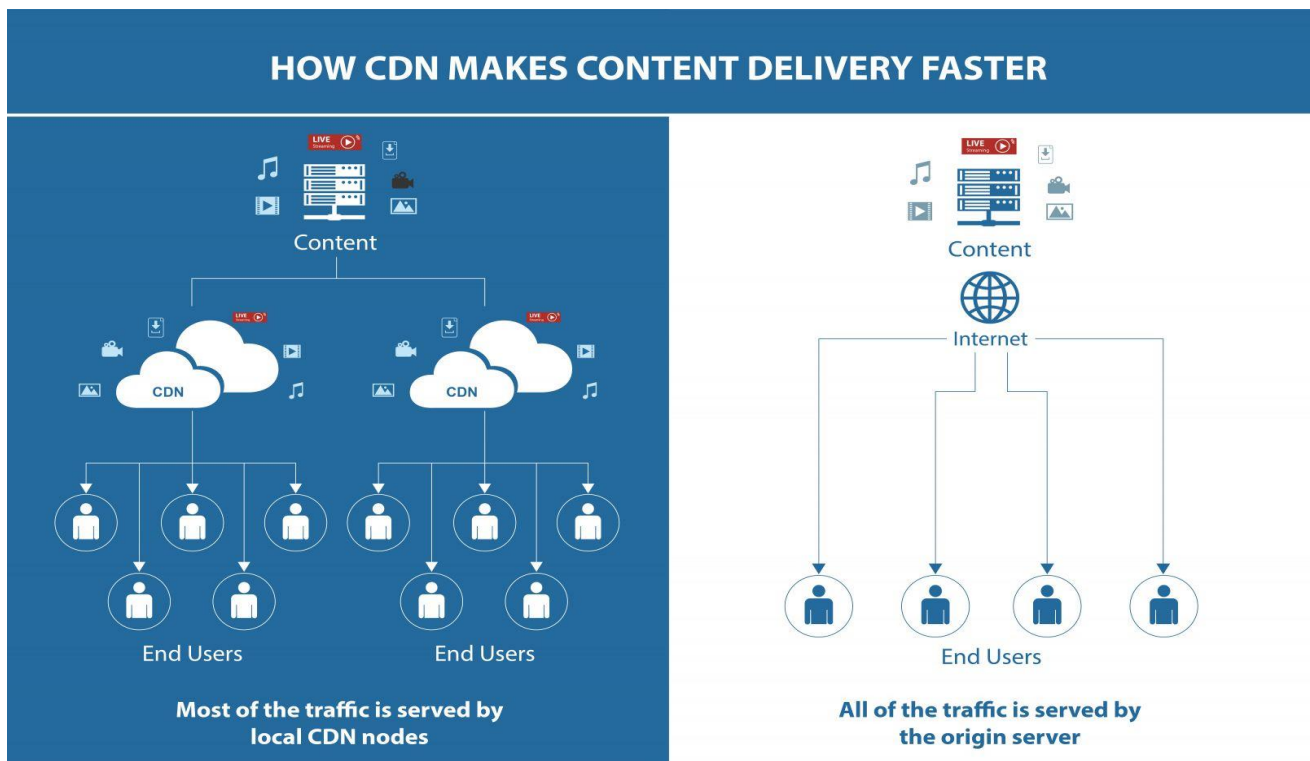
➤ **Strategies:**

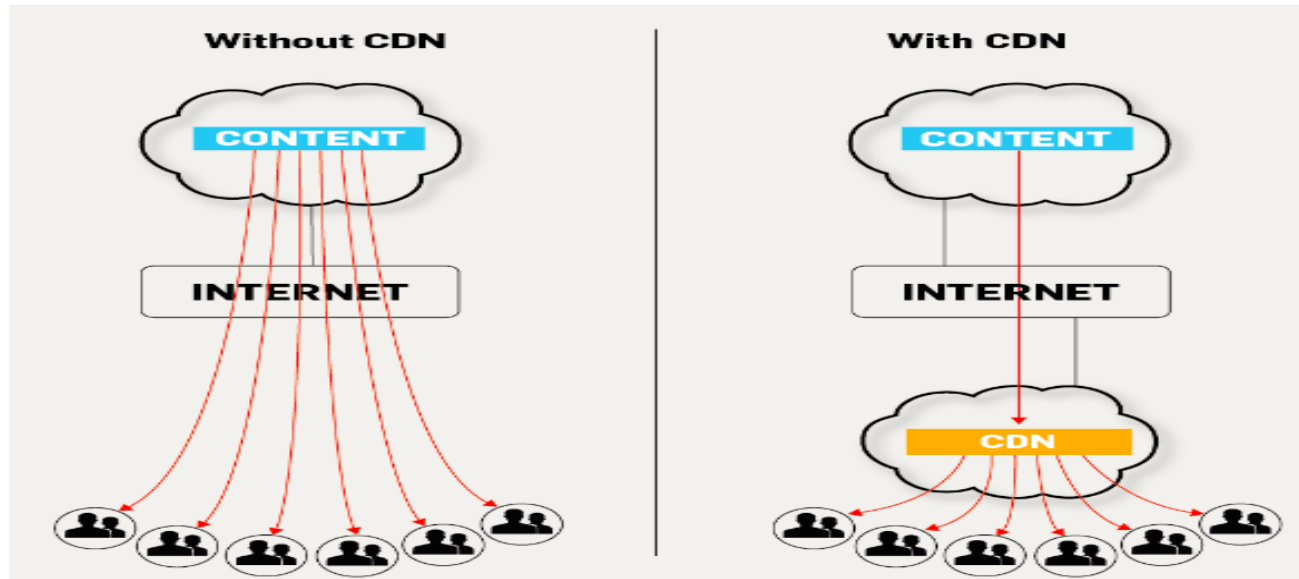
- Compress images using tools like TinyPNG or ImageOptim.
- Use modern formats like WebP for better compression.
- Implement responsive images with `srcset` to serve appropriately sized images.
- Use lazy loading to defer the loading of offscreen images until needed.



3. Use Content Delivery Networks (CDNs):

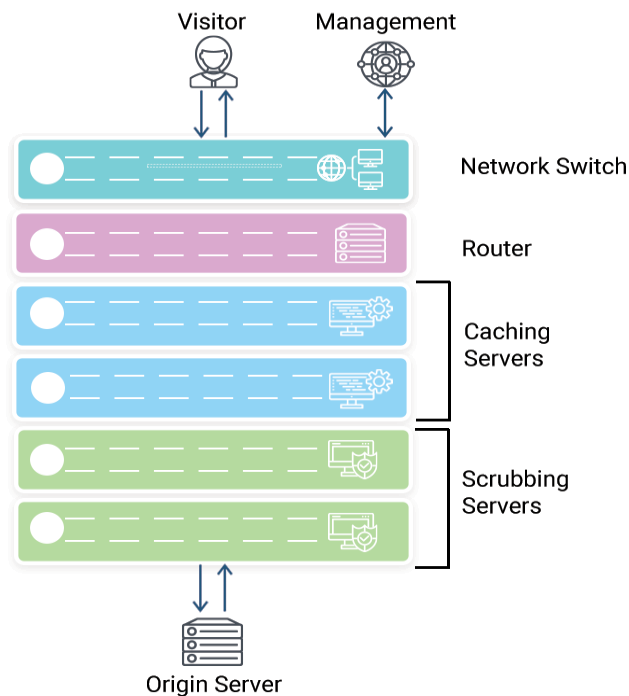
Content delivery networks (CDNs) improve site performance by using a distributed network of servers to deliver resources to users. Because CDNs reduce server load, they reduce server costs and are well-suited to handling traffic spikes.





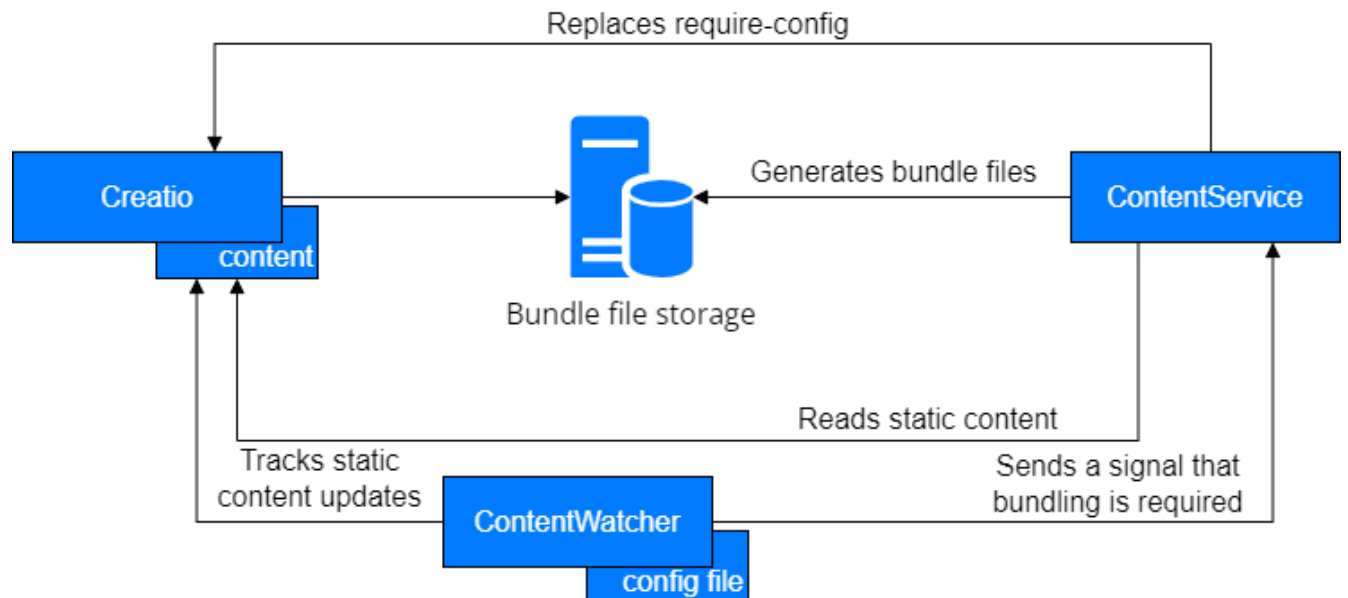
- **Objective:** Reduce latency and improve load times by caching content globally.
- **Strategies:**
 - Choose a CDN provider (e.g., Cloudflare, Akamai).
 - Configure CDN to cache static assets like images, CSS, and JavaScript files.

HOW CONTENT DELIVERY NETWORK FUNCTIONS



4. Minify and Bundle Files:

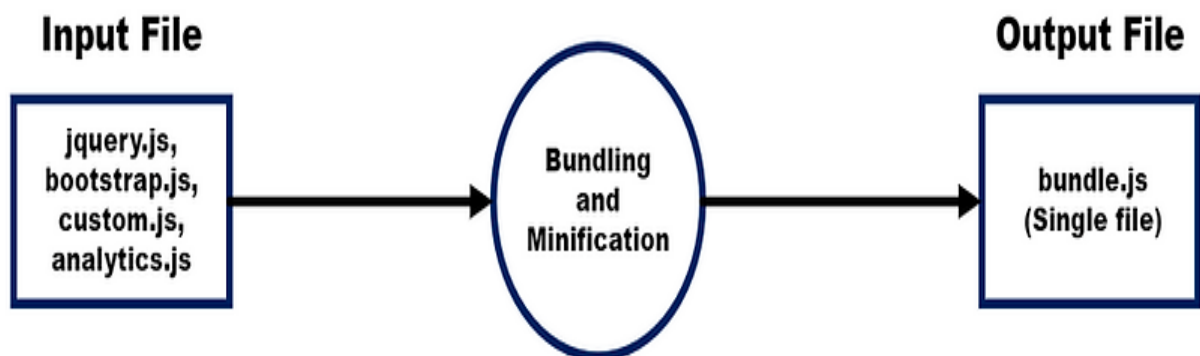
Bundling and minification are two techniques you can use in ASP.NET 4.5 to improve request load time. Bundling and minification improves load time by reducing the number of requests to the server and reducing the size of requested assets (such as CSS and JavaScript.)



➤ **Objective:** Reduce file sizes and number of requests.

➤ **Strategies:**

- Minify CSS, JavaScript, and HTML using tools like UglifyJS, CSSNano.
- Bundle multiple files into one using build tools like Webpack or Gulp.



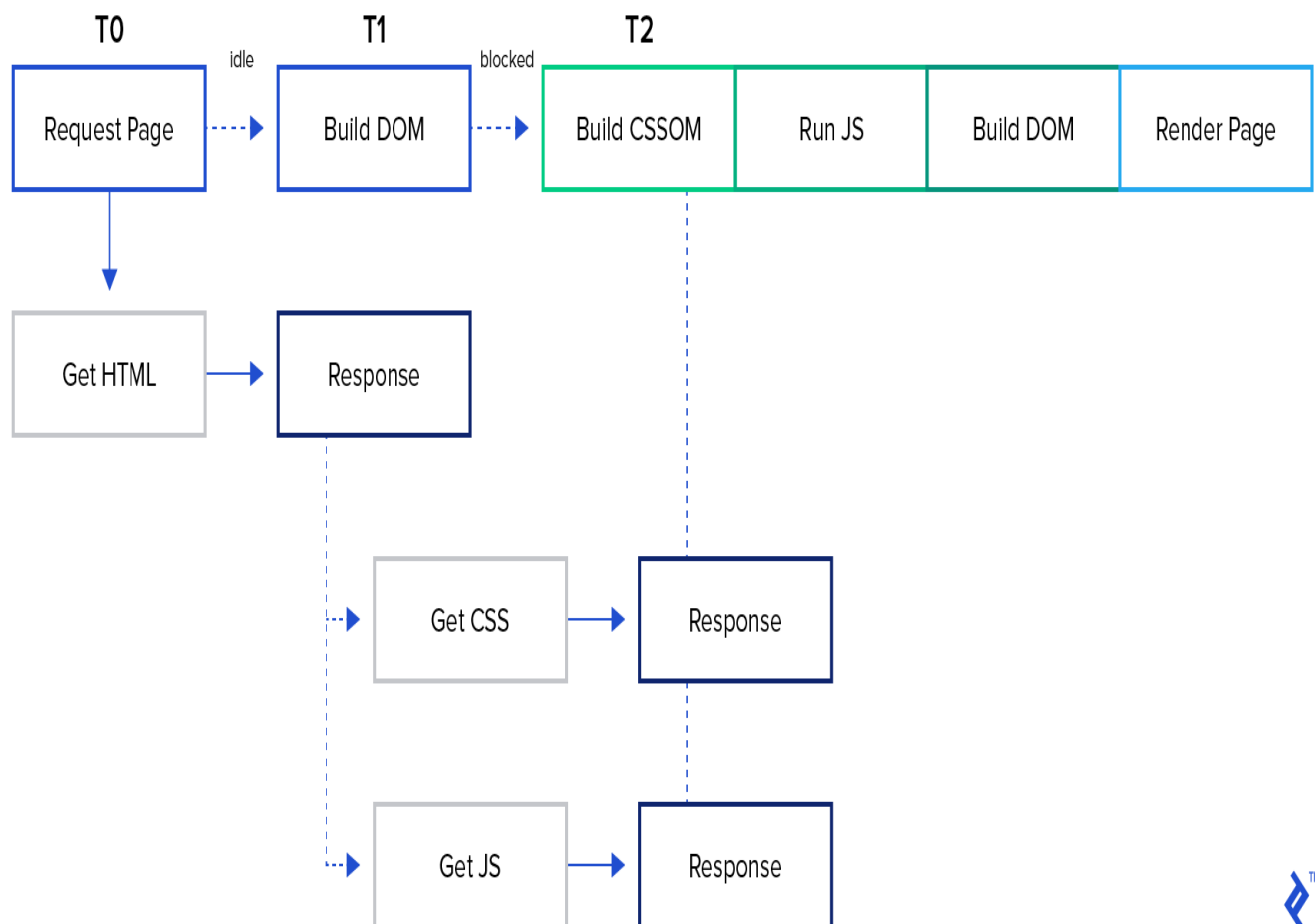
5. Optimize CSS and JavaScript:

- ✚ Optimize CSS and JS files—Reduce the time required to load CSS and JavaScript (JS) files by configuring Adobe Commerce to merge, minify, and bundle separate files into a single file. Eliminate render-blocking resources—Consider delivering critical JS and CSS features inline and deferring all non-critical JS/CSS styles.
- ✚ Make JS load faster in the browser. Reduce how much JS you use to build your page. Minify all JS resources, and use minified third-party JS. ...
- ✚ Only load JS when it's needed. Eliminate dead JS code. Split JS files to deliver essential components.

➤ **Objective:** Avoid blocking page rendering and improve load times.

➤ **Strategies:**

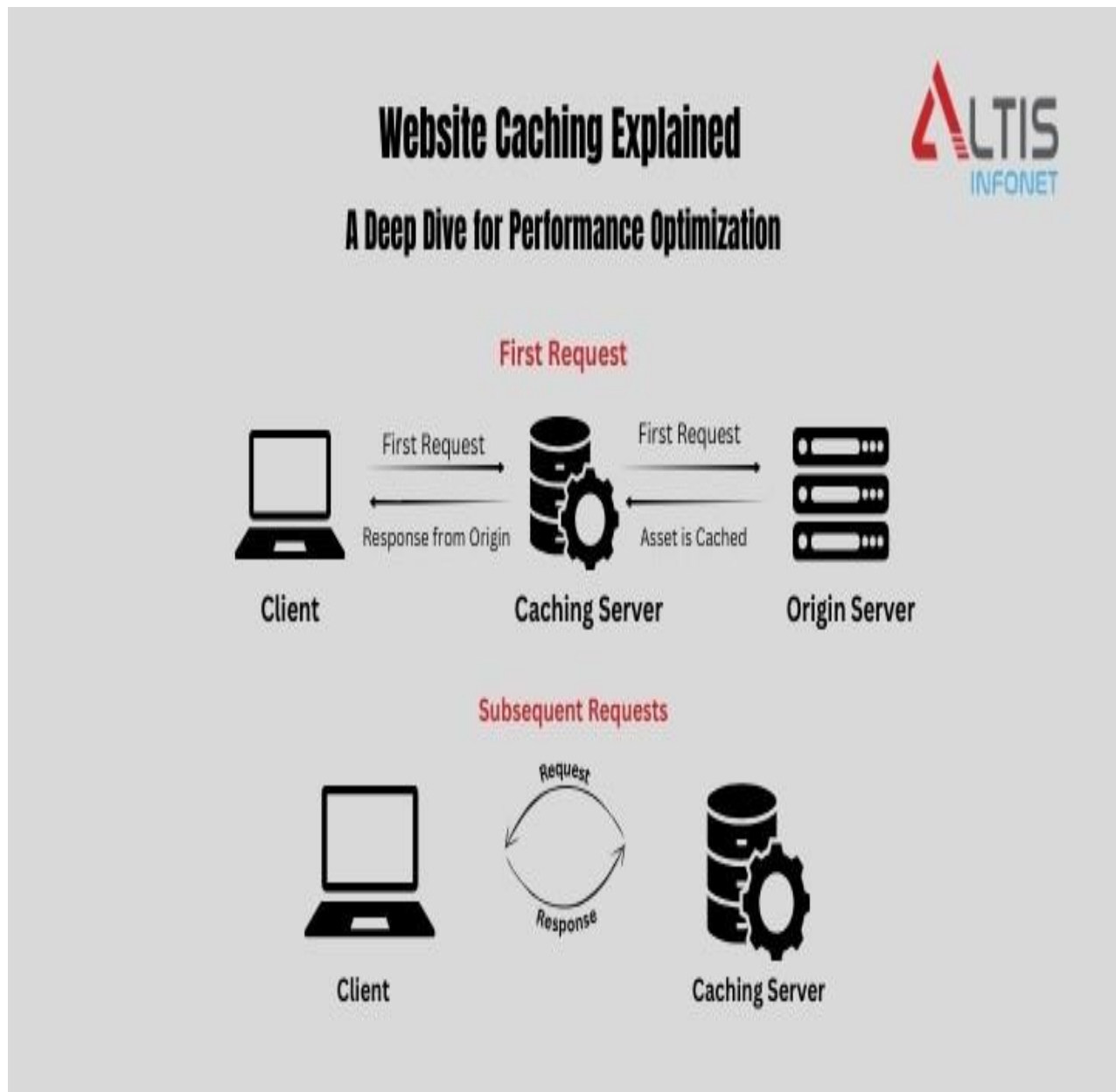
- Load CSS files asynchronously with `rel="preload"` or `media="print"`.
- Defer JavaScript loading with `async` or `defer` attributes.



6. Browser Caching:

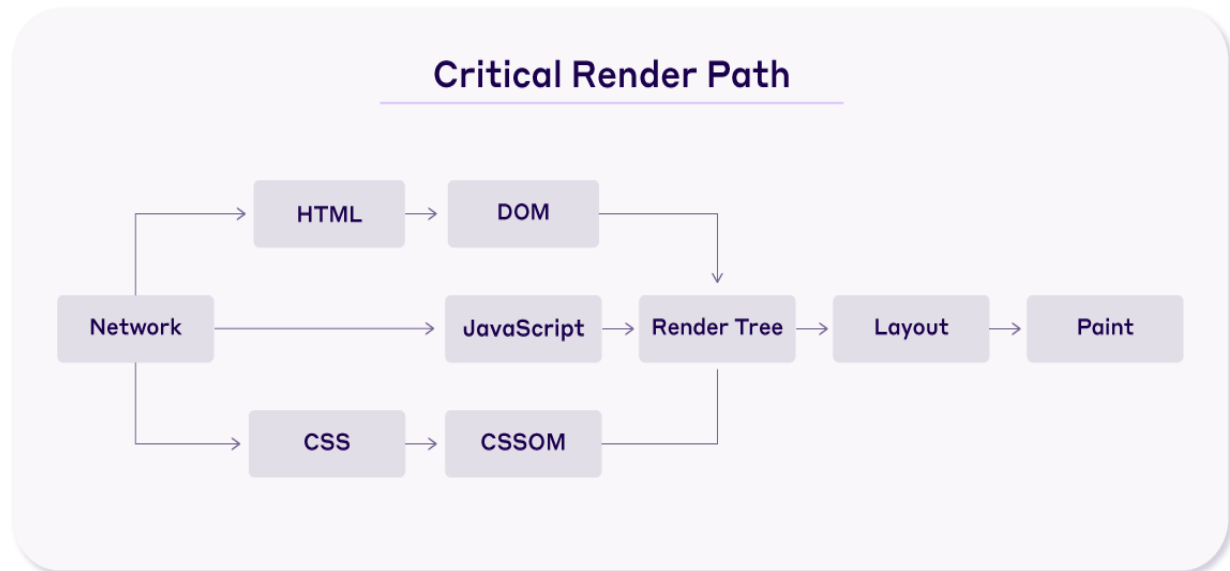
Browser caching is a web performance optimization technique that involves storing and reusing web page assets on a user's device. These assets can range from images, stylesheets, JavaScript files, fonts, to even entire web pages.

- **Objective:** Reduce the need for re-downloading static assets.
- **Strategies:**
 - Set proper cache headers using `Cache-Control`, `Expires`, and `ETag`.
 - Configure server settings to leverage browser caching.



7. Critical Rendering Path:

- **Objective:** Improve page rendering speed by optimizing the order of resource loading.
- **Strategies:**
 - Minimize CSS and JavaScript that blocks rendering.
 - Prioritize above-the-fold content to load first.

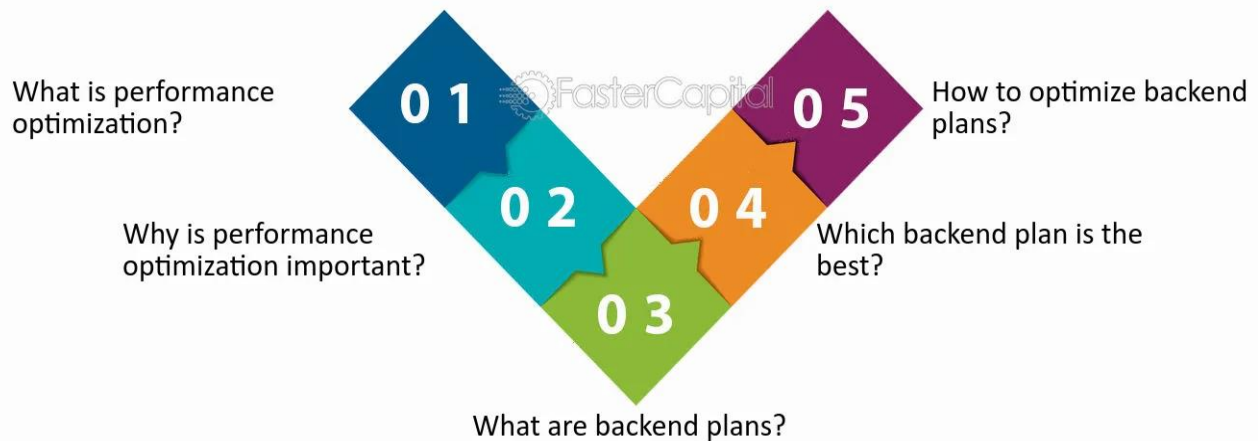


B. Back-End Performance Optimization

- 🔧 Backend optimization refers to the process of improving the server-side performance of a web application. This involves various techniques aimed at reducing latency, increasing efficiency, and ensuring scalability.
- 🔧 There are three elements of the back-end of a website, including the server, application, and database, and these three parts are what back-end developers focus on in their work.
- 🔧 Performance Optimization utilizes various techniques, such as code optimization, system tuning, and load balancing, among others. It primarily focuses on improving the computational efficiency of a system by reducing resource usage and minimizing latencies.

Definition: Enhancing the efficiency and speed of server-side processes and database interactions.

Understanding Performance Optimization and Backend Plans



❖ Key Areas to Focus On:

1. **Database Optimization:**

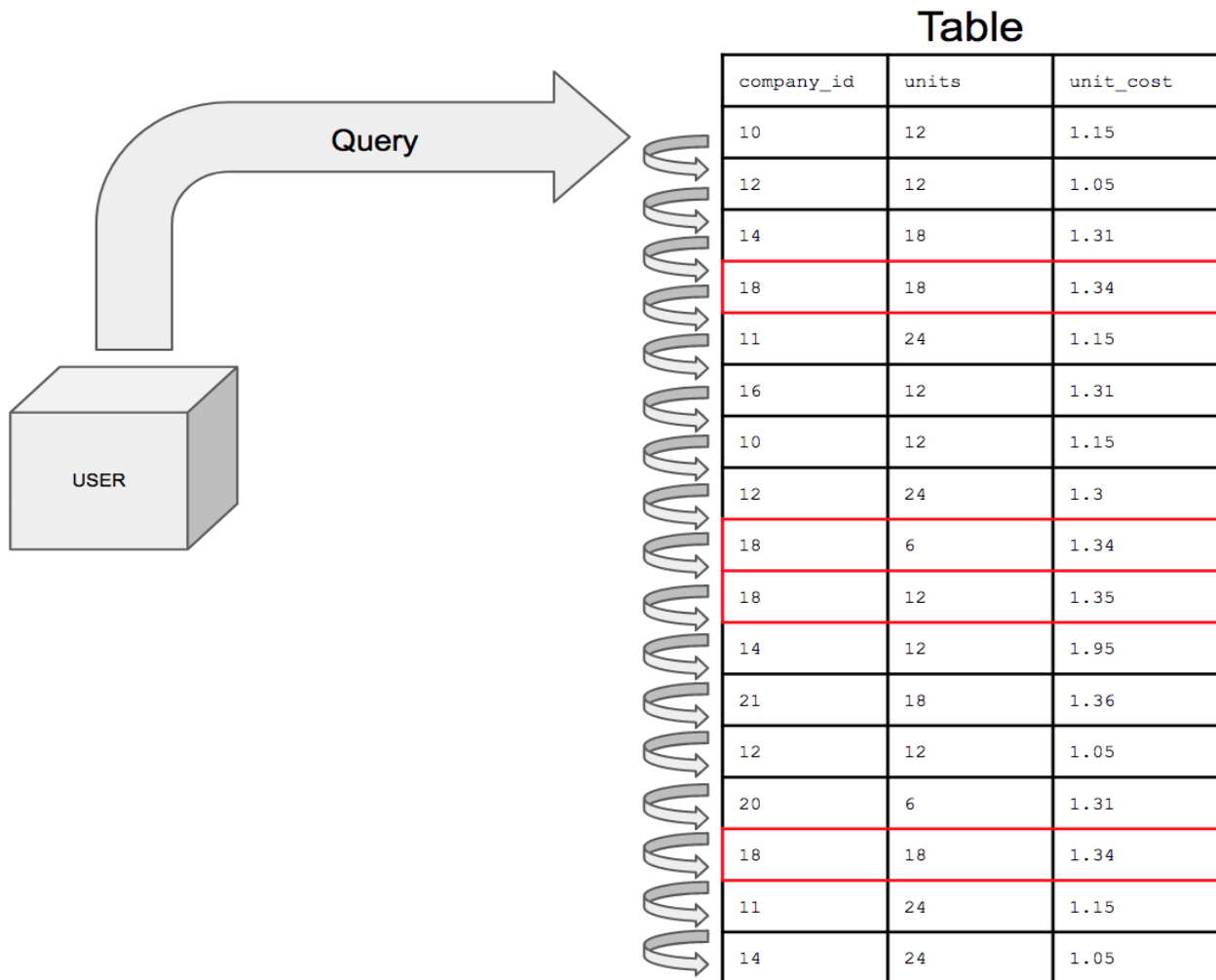
Database performance optimization is the process of improving the performance of a database system, ensuring that it can handle increasing amounts of data, user requests, and concurrent transactions without experiencing slow response times or system failures.

- Improve indexing strategies
- Identify access to database.
- Evaluate connection capacity
- Optimize Queries.
- Database Performance Resources.
- Check your database server.

➤ **Objective:** Speed up database operations and reduce load times.

➤ **Strategies:**

- Use indexing to speed up query retrieval.
- Optimize queries to avoid full table scans and reduce complexity.
- Design schema efficiently to minimize redundant data and optimize relationships.

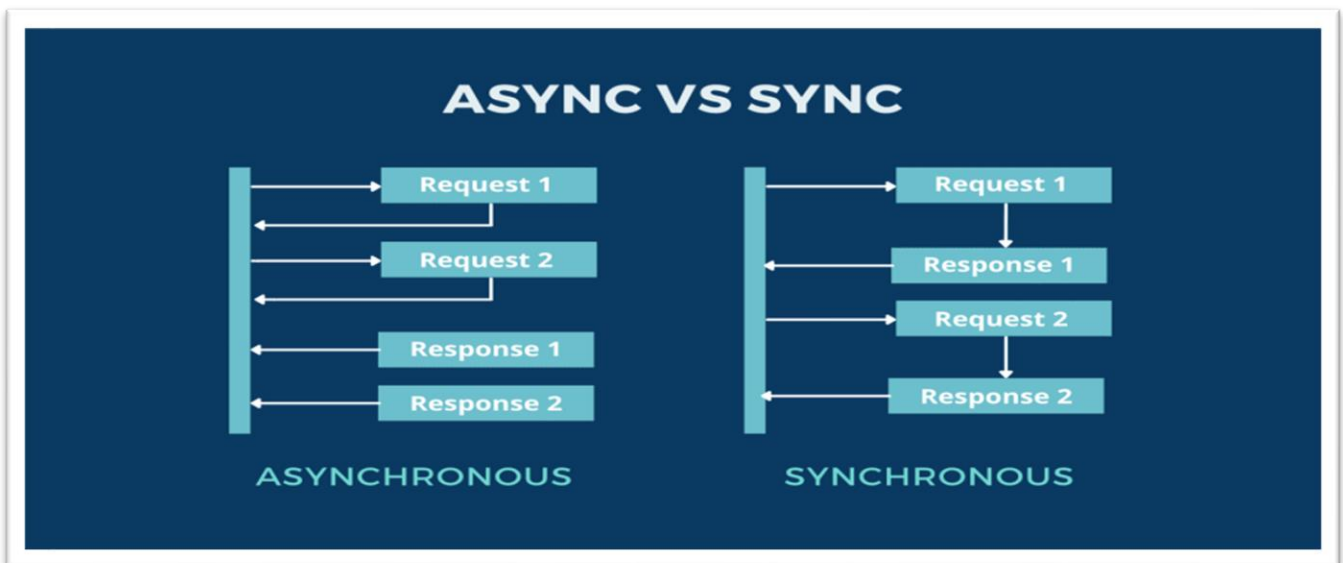


2. Optimize Server-Side Code:

- Select the Right Application for Server Optimization.
 - Enable Caching.
 - Setup a Quick Reverse Proxy.
 - Defragment Database Tables & Optimize Server Settings.
 - Fix DNS Query Response Time.
- **Objective:** Ensure server-side code runs efficiently and quickly.
- **Strategies:**
- Write efficient algorithms and avoid unnecessary computations.
 - Profile code using tools to identify and fix bottlenecks (e.g., New Relic, Datadog).

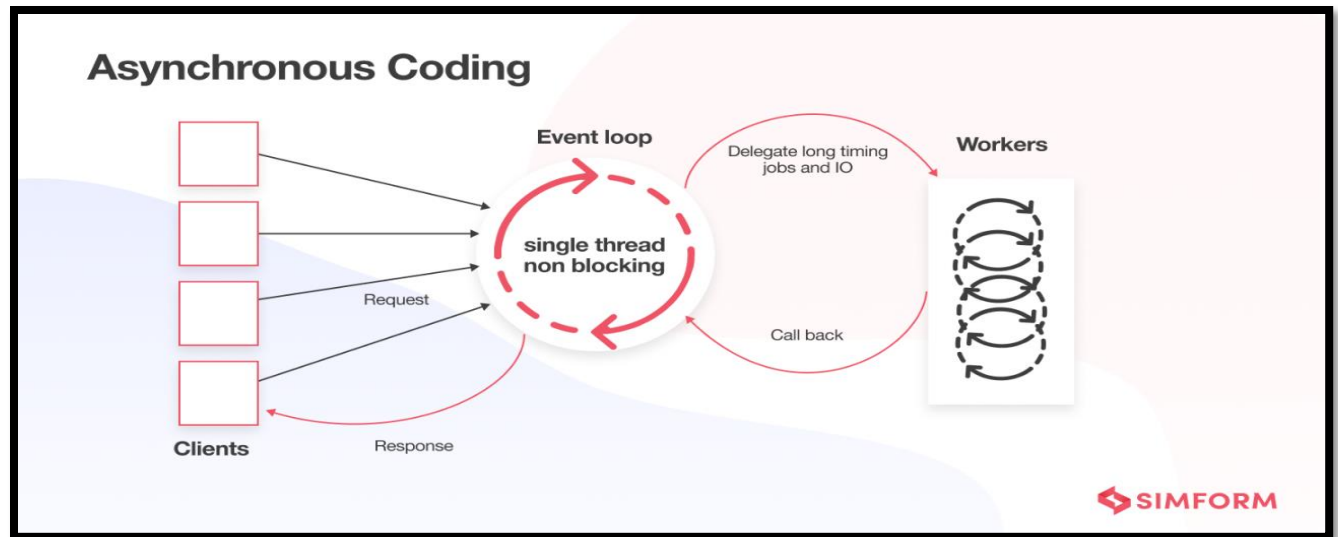
3. Use Asynchronous Processing:

- ✚ Asynchronous processing allows tasks to be executed independently of the main program flow. This means that tasks can run concurrently, enabling the system to handle multiple operations simultaneously.
 - ✚ Unlike synchronous processing, where tasks are completed one after another, asynchronous processing helps in reducing idle time and improving efficiency. This approach is particularly advantageous for tasks that involve waiting, such as I/O operations or network requests. By not blocking the main execution thread, asynchronous processing ensures that systems remain responsive and performant.
- **Objective:** Improve responsiveness and handle background tasks efficiently.
- **Strategies:**
- Offload tasks like email sending or file processing to background workers using tools like Celery or RabbitMQ.



Key features of asynchronous processing include:

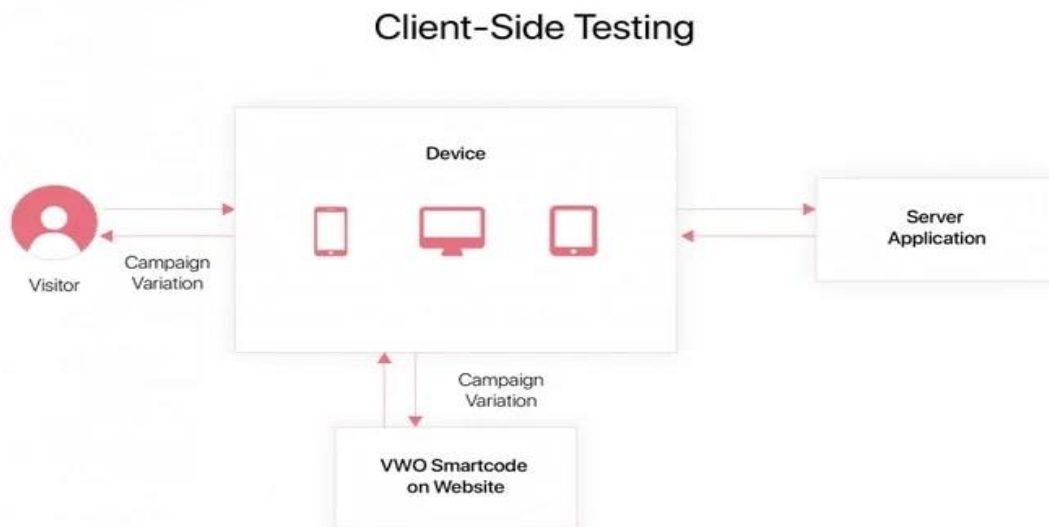
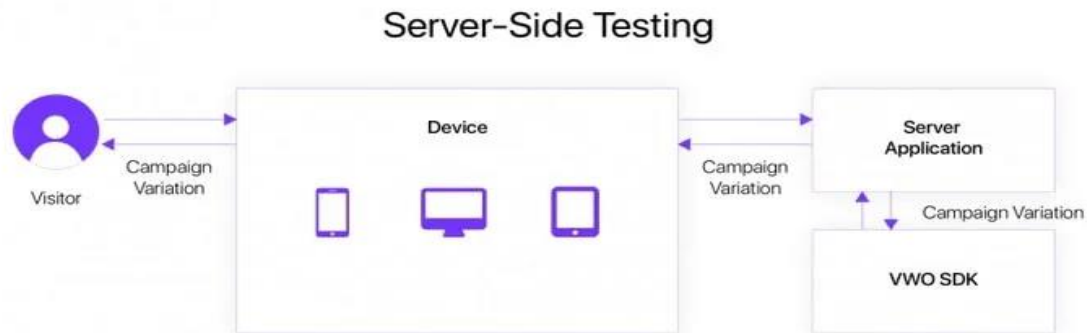
- **Concurrent Execution:** Tasks can run at the same time without waiting for each other to complete.
- **Non-blocking Operations:** The main program continues to run while waiting for other tasks to finish.
- **Improved Efficiency:** Reduces idle time by utilizing system resources more effectively.
- **Enhanced Responsiveness:** Applications remain responsive even when performing long-running tasks.
- **Ideal for I/O Operations:** Particularly beneficial for tasks that involve waiting, like file reading or network communication.



4. Load Testing and Profiling:

Performance **Profiling** is a 4-stage process, which involves identifying the qualities required to be successful in your sport:

- Stage 1: Ranking and defining the most important qualities
 - Stage 2: Plotting your own performance profile.
 - Stage 3: Generating action points.
 - Stage 4: Identifying barriers.
- **Profiling** helps in fine tuning the application while performance **testing** helps identifying performance bottlenecks like application slowness during peak load or server failure during load and so on. Profiling helps in optimizing the code by providing granular data for a process, thread, method, sql, etc
 - This difference is important because performance **testing** checks how long an app can run and how well it can grow in real-life situations that are simulated. **Profiling**, on the other hand, gives you more information about specific code problems and performance issues.
- **Objective:** Understand and prepare for high traffic scenarios.
 - **Strategies:**
 - Conduct load tests using tools like JMeter or Locust to simulate traffic and identify weak points.
 - Use profiling tools to monitor server performance and identify bottlenecks.



5. Resource Management:

Resource management is a series of processes and techniques used to ensure you have all the necessary resources to complete a project or meet business objectives. It also focuses on making the most efficient use of those resources by eliminating waste for more profits and a high return on investment (ROI).

- **Objective:** Optimize server resources for better performance.
- **Strategies:**
 - Monitor and manage CPU and memory usage.
 - Consider upgrading server hardware or utilizing cloud-based solutions for scalability.

5 Steps

to optimize **Project Resource Management**

01 UNDERSTAND CURRENT RESOURCE CAPABILITIES

02 UNDERSTAND HOW ROLES ARE UTILIZED TODAY

03 ANALYZE FUTURE NEEDS & RESOURCE DEMAND

04 BUILD A RESOURCE PLANNING MODEL

05 USE DATA TO EVALUATE RESOURCE MANAGEMENT EFFICIENCY



3. Caching Strategies

Definition: Storing copies of files or data in temporary storage to reduce access time for frequently requested information.

Introduction to Caching Strategies

Fully Associative Caches

Set-Associative Caches

Replacement Policies

Direct-Mapped Caches

Cache Coherency

Introduction to Caching Strategies

Case Studies and Tips



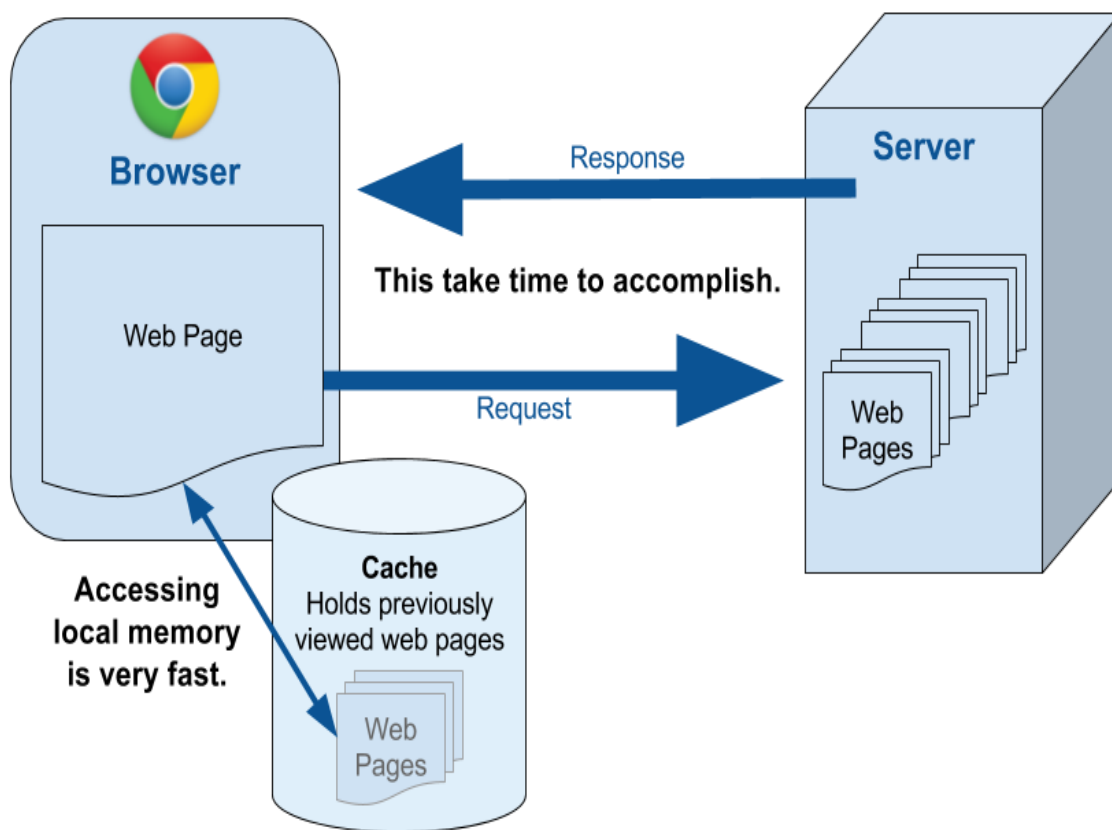
Implementing caching strategies serves frequently requested data from memory instead of querying the database or performing complex computations each time. In-Memory Caching: Tools like Redis and Memcached store frequently accessed data in memory, enabling faster data retrieval.

Key Types of Caching:

1. Browser Caching:

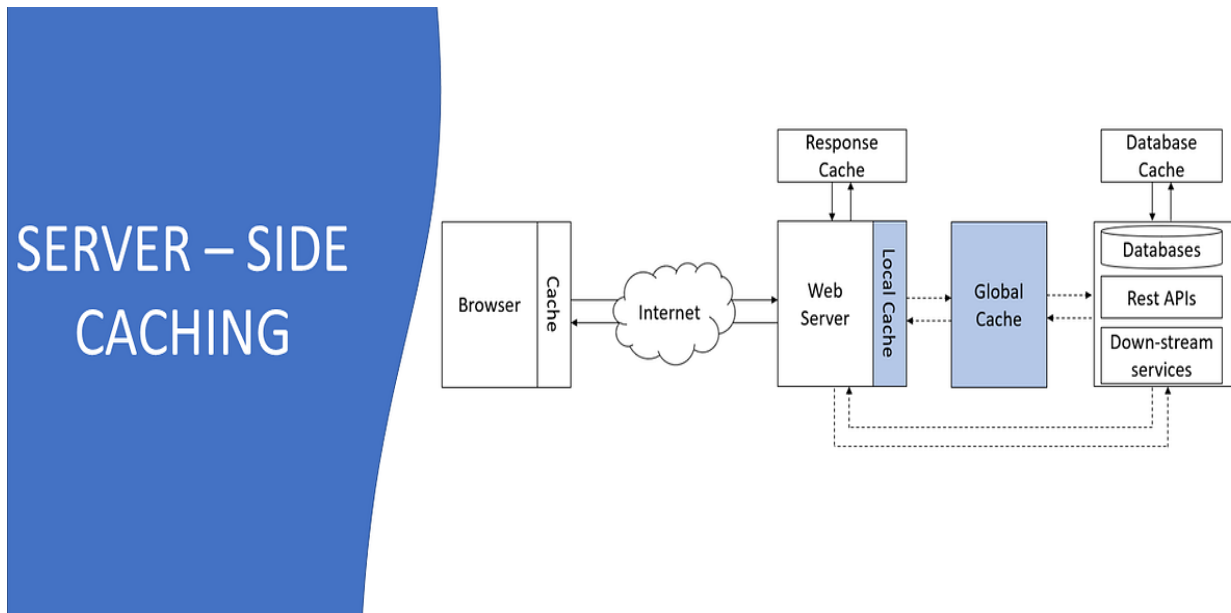
- **Objective:** Reduce server requests and improve load times for returning visitors.
- **Strategies:**
 - Store static resources (CSS, JavaScript, images) in the browser cache.
 - Set cache-control headers to specify caching rules.

The Browser Cache



2. Server-Side Caching:

- **Objective:** Speed up dynamic content generation and reduce database load.
- **Strategies:**
 - Implement object caching for frequently accessed data (e.g., using Redis or Memcached).
 - Use page caching to store the rendered output of dynamic pages.

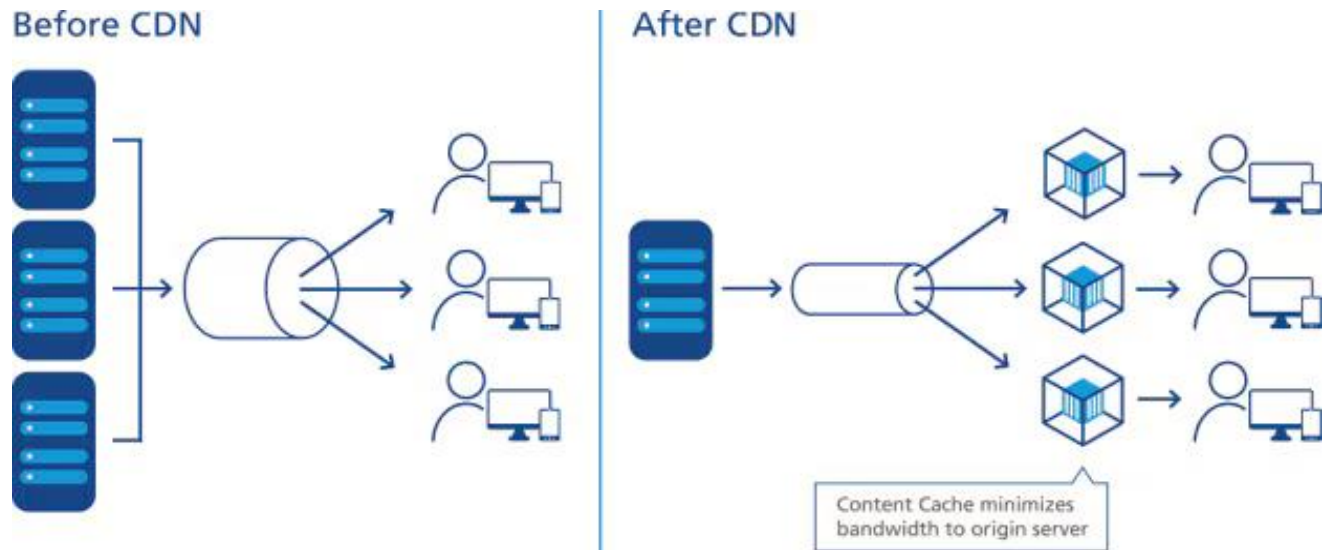


3. Content Delivery Network (CDN) Caching:

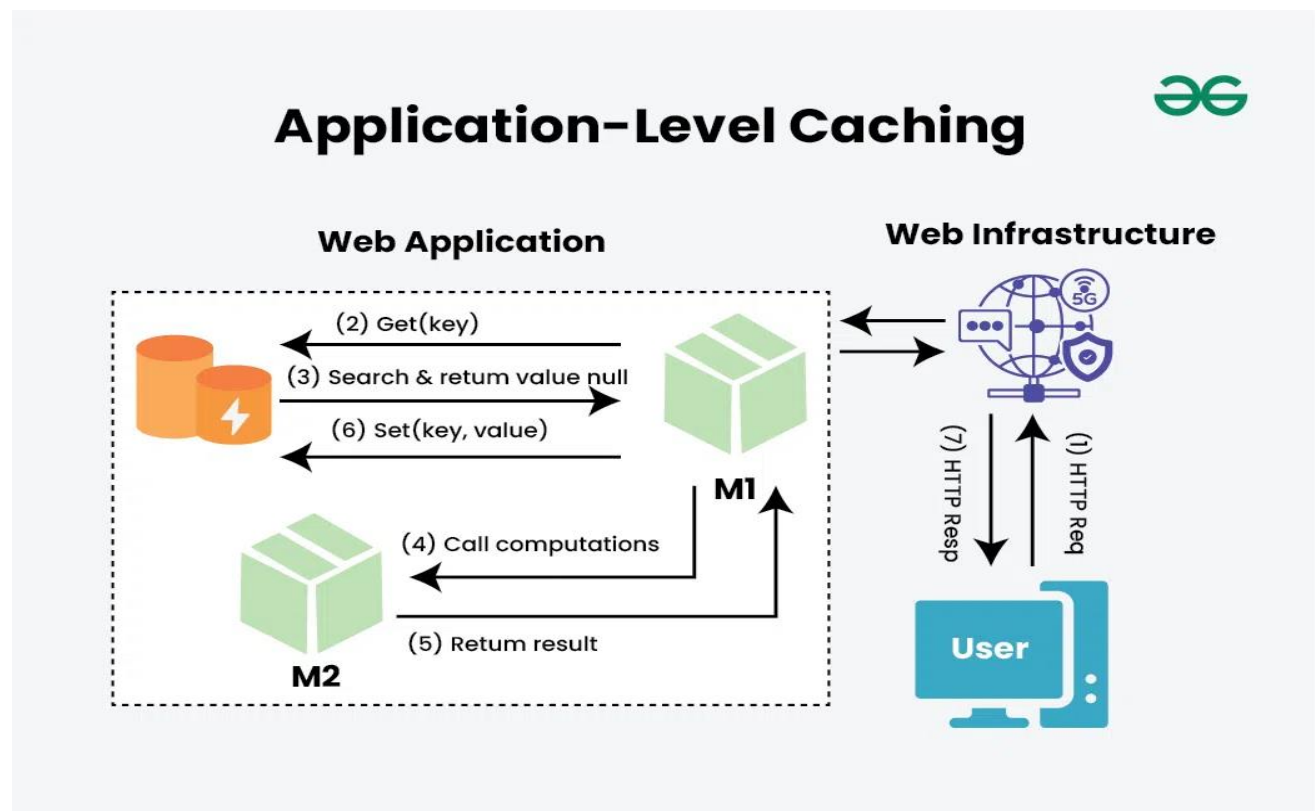
Implementing a CDN (Content Delivery Network) can significantly enhance your web application by reducing the need for multiple back-end servers. This shift not only results in faster loading times for end-users but also streamlines content delivery.

Caching is at the heart of content delivery network (CDN) services. Similar to how browser caching stores files on a hard drive, where they can be more rapidly accessed, a CDN moves your website content to powerful proxy servers optimized for accelerated content distribution.

- **Objective:** Cache static content on edge servers close to users to improve performance.
- **Strategies:**
 - Configure CDN to cache and deliver static assets like images and scripts.



4. Application-Level Caching:



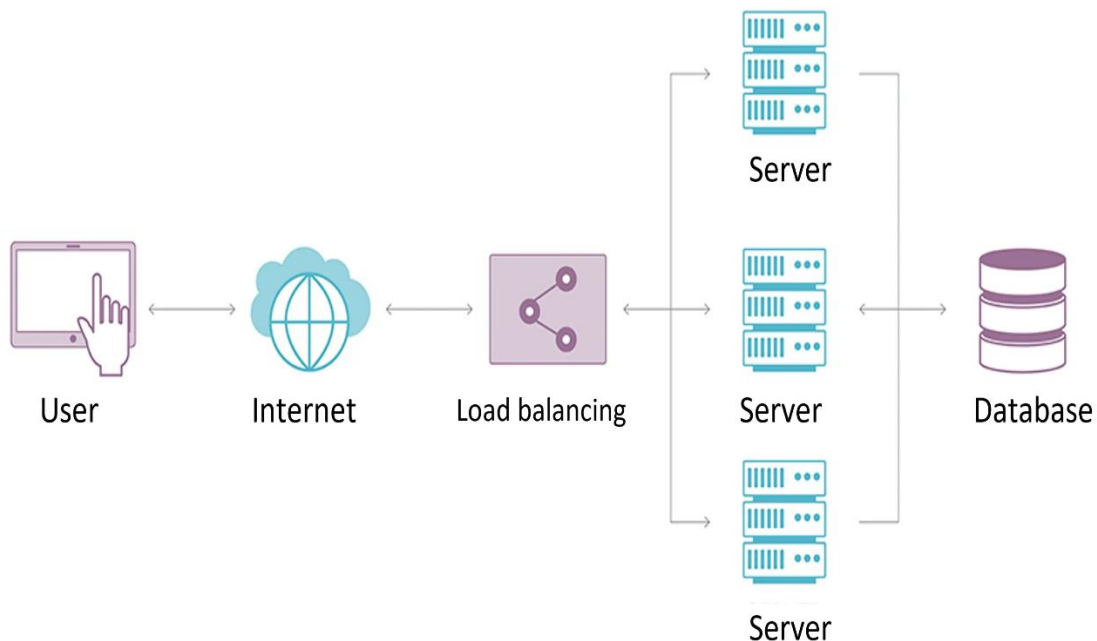
- Application-level caching is useful for data that is specific to the application logic, such as user sessions, configuration settings, or business rules.
- Application-level caching is fast and easy to set up, but it also has some limitations, such as memory consumption, scalability issues, and lack of persistence.

- **Objective:** Enhance performance by caching frequently accessed application data.
- **Strategies:**
 - Use caching libraries or services (e.g., Redis, Memcached) for data that is costly to compute or fetch.

5. Load Balancing and Scalability:

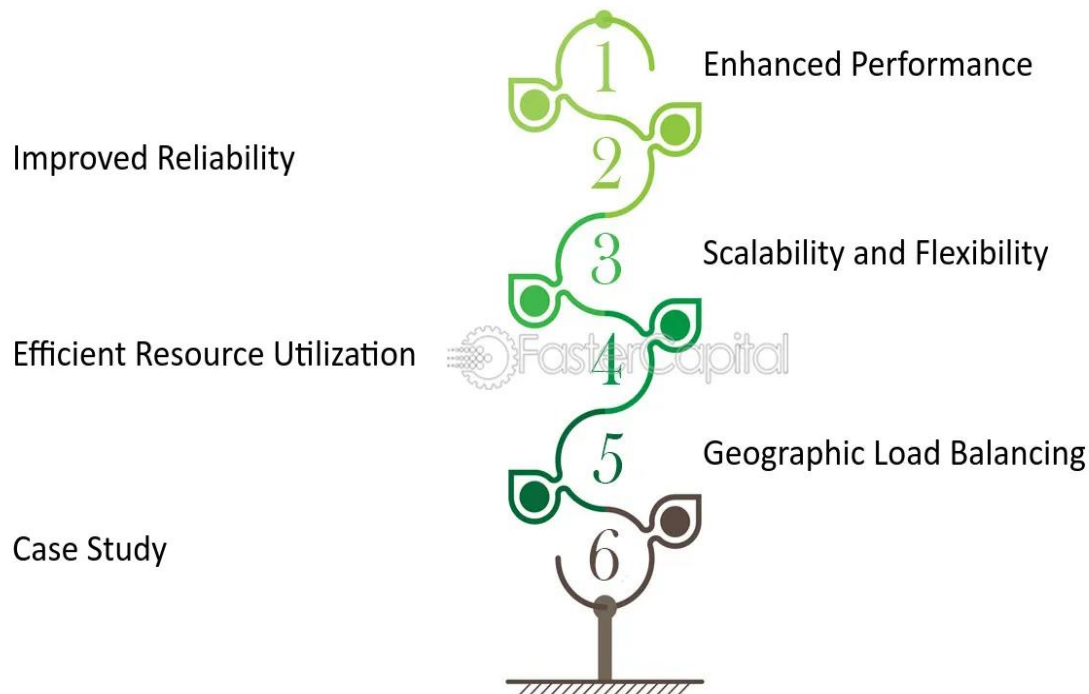
- ✚ Load balancing distributes network traffic dynamically across a network of resources that support an application. A load balancer is the device or service that sits between the user and the server group and acts as an invisible facilitator, ensuring that all resource servers are used equally.
- ✚ Load balancing directs and controls internet traffic between the application servers and their visitors or clients. As a result, it improves an application's availability, scalability, security, and performance.

How Load balancing works?



Definition: Load balancing distributes traffic across multiple servers, while scalability involves expanding resources to handle increased load.

Benefits of Load Balancing for Scalability



Key Concepts:

1. Load Balancing:

➤ Types of Load Balancers:

- **Hardware-Based:** Physical devices for managing traffic (e.g., F5 BIG-IP).
- **Software-Based:** Applications that distribute traffic (e.g., Nginx, HAProxy).

➤ Algorithms:

- **Round Robin:** Distribute requests sequentially.
- **Least Connections:** Send requests to the server with the fewest active connections.
- **IP Hash:** Route requests based on the client's IP address.

2. Scalability:

➤ Vertical Scaling:

- **Definition:** Increase power (CPU, RAM) of existing servers.
- **Limitation:** Limited by hardware constraints.

➤ Horizontal Scaling:

- **Definition:** Add more servers to handle increased load.
- **Approach:** Implement load balancing to distribute traffic.

➤ Auto-Scaling:

- **Definition:** Automatically adjust resources based on traffic.
- **Implementation:** Use cloud services like AWS Auto Scaling or Azure Scale Sets.

3. Stateless Applications:

- **Objective:** Simplify scaling by making applications stateless.
- **Strategies:**
 - Ensure that each request is independent and does not rely on server-side session data.

